

React Lab: Props, Default Props, State vs Props, and ReactDOM.render()

Hands-on Lab Solution — "shoppingapp" (OnlineShopping & Cart Components)

Objectives

- Define Props
- Explain Default Props
- Identify the differences between State and Props
- Explain ReactDOM.render()

Prerequisites

- Node.js
- NPM
- Visual Studio Code

1. What are Props?

Props (short for “properties”) are read-only inputs passed from a parent component to a child component. They allow data to flow one-way, from parent to child, and let a single component be reused with different values. A component must never modify its own props — they are owned and controlled by the parent that renders it.

In the class-based syntax used in this lab, props are accessed inside a component through `this.props`. For example, in the Cart component below, `this.props.itemname` and `this.props.price` are the values supplied by the parent component, OnlineShopping.

2. What are Default Props?

`defaultProps` is a static property that lets a component specify fallback values for any props that the parent does not explicitly pass. If the parent omits a prop, React automatically substitutes the value defined in `defaultProps` instead of the prop being undefined. This protects the component from breaking when data is missing and documents the expected shape of its props.

In this lab, the Cart component defines default values for `itemname` and `price`, so if OnlineShopping ever renders a `<Cart />` without supplying those props, sensible defaults are shown instead of a blank or broken row.

3. Differences Between State and Props

Props	State
Passed into a component from its parent.	Managed and owned internally by the component itself.
Read-only — cannot be changed by the component that receives them.	Mutable — changed using <code>this.setState()</code> (class components) or a state setter (hooks).
Used to configure a component and pass data down the tree.	Used to track data that changes over time, such as user input or toggles.

Props	State
Changing a prop's source value in the parent triggers a re-render of the child.	Calling <code>setState()</code> triggers a re-render of the component that owns the state.
Accessed via <code>this.props</code> in class components.	Accessed via <code>this.state</code> in class components.

In this lab, the array of five Cart items is held in the state of OnlineShopping (`this.state.items`), because OnlineShopping owns and controls that data. Each individual Cart item's `itemname` and `price` are then passed down as props, because Cart does not own that data — it simply displays whatever it is given.

4. What is ReactDOM.render()?

`ReactDOM.render()` is the method that mounts a React element (or component tree) into a real DOM node in the HTML page, and keeps it updated whenever the underlying data changes. It takes two arguments: the React element to render, and the DOM container to render it into.

```
ReactDOM.render(<OnlineShopping />, document.getElementById('root'));
```

This single call is what bridges React's virtual component tree to the actual index.html page created by Create React App, which contains a `<div id="root"></div>` element. Note: from React 18 onward, `ReactDOM.render()` is deprecated in favor of `ReactDOM.createRoot(container).render(<App />)`; both are shown in Step 5 below so the concept is clear either way.

Lab Steps

Step 1: Create the React application

Open a terminal and run the following commands to scaffold the project:

```
npx create-react-app shoppingapp
cd shoppingapp
npm start
```

Open the shoppingapp folder in Visual Studio Code. Inside the `src` folder, create two new files: `Cart.js` and `OnlineShopping.js`.

Step 2: Create the Cart class component

Cart is a presentational component. It receives `itemname` and `price` as props and simply displays them. `defaultProps` is added so the component still renders sensibly even if a prop is missing.

src/Cart.js

```
import React, { Component } from 'react';

class Cart extends Component {
  render() {
    return (
      <div style={{ border: '1px solid #ccc', borderRadius: '6px',
        padding: '10px', margin: '8px 0', width: '260px' }}>
        <p><strong>Item Name:</strong> {this.props.itemname}</p>
        <p><strong>Price:</strong> ${this.props.price}</p>
      </div>
    );
  }
}
```

```

    </div>
  );
}
}

// Default Props: used only if a prop is not passed by the parent
Cart.defaultProps = {
  itemname: 'Unknown Item',
  price: 0
};

export default Cart;

```

Step 3: Create the OnlineShopping class component

OnlineShopping owns the data. Its constructor initializes state with an array of five cart items. The render() method loops through this.state.items using .map() and renders one <Cart /> component per item, passing itemname and price down as props.

src/OnlineShopping.js

```

import React, { Component } from 'react';
import Cart from './Cart';

class OnlineShopping extends Component {
  constructor(props) {
    super(props);
    // State: owned and managed by OnlineShopping itself
    this.state = {
      items: [
        { id: 1, itemname: 'Laptop',      price: 55000 },
        { id: 2, itemname: 'Mobile Phone', price: 15000 },
        { id: 3, itemname: 'Headphones',  price: 2000  },
        { id: 4, itemname: 'Keyboard',    price: 800   },
        { id: 5, itemname: 'Mouse',       price: 500   }
      ]
    };
  }

  render() {
    return (
      <div style={{ fontFamily: 'Arial, sans-serif', padding: '20px' }}>
        <h1>Online Shopping Cart</h1>
        {this.state.items.map((item) => (
          // Props: itemname and price are passed down to each Cart
          <Cart key={item.id} itemname={item.itemname} price={item.price} />
        ))}
      </div>
    );
  }
}

export default OnlineShopping;

```

Step 4: Update App.js

src/App.js

```
import React from 'react';
import OnlineShopping from './OnlineShopping';

function App() {
  return (
    <div className="App">
      <OnlineShopping />
    </div>
  );
}

export default App;
```

Step 5: Render with ReactDOM

This is the entry point of the app. It mounts the App component into the root div of public/index.html.

src/index.js (React 17 and earlier — classic ReactDOM.render)

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

src/index.js (React 18+ — createRoot, generated by current Create React App)

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Step 6: Run the application

```
npm start
```

This opens the app at <http://localhost:3000>. The page displays the heading “Online Shopping Cart” followed by five bordered boxes, one per Cart item, each showing its Item Name and Price — exactly the five items initialized in the OnlineShopping component's state.

Expected Output

Online Shopping Cart

Item Name: Laptop	Price: \$55000
Item Name: Mobile Phone	Price: \$15000
Item Name: Headphones	Price: \$2000
Item Name: Keyboard	Price: \$800
Item Name: Mouse	Price: \$500

Summary

- Props carry data one-way from a parent (OnlineShopping) to a child (Cart) and cannot be changed by the child.
- Default Props (Cart.defaultProps) supply fallback values when a parent does not pass a prop.
- State belongs to and is managed by the component that declares it; here OnlineShopping owns the items array in this.state, while Cart only receives props.
- ReactDOM.render() (or ReactDOM.createRoot().render() in React 18+) mounts the top-level React component tree into the real DOM element with id="root".